



WOLLO University

Department of software engineering

Blueprint based exit material for

Requirements engineering &

Architectural design

Prepared by:

@Neyatibeb

KOMBOLCHA, Ethiopia

2015 E.C

Requirement engineering

❖ Important of requirement engineering

- To make the problem that is being stated clear and complete, and to ensure that the solution is correct, reasonable, and effective
- provides a vision of the final software
- helps in defining the scope of the software
- helps in perceiving the cost of the final software
- It also helps in perceiving the schedule up to which the software will be delivered to the customer.
- Improved communication and collaboration
- Increased visibility and traceability
- Enhanced compliance
- Reduced risk of project failure

Requirement engineering is the process of gathering, documenting, and managing the requirements for a software system. It is a critical part of the software development lifecycle, as it helps to ensure that the system meets the needs of the users and stakeholders.

Here are some of the most important reasons why requirement engineering is important:

- To ensure that the system meets the needs of the users and stakeholders: The requirements document is a contract between the development team and the stakeholders. It ensures that the system will meet the needs of the stakeholders, and that the development team will not build something that is not needed.
- To reduce the risk of project failure: A well-defined and well-documented set of requirements can help to reduce the risk of project failure. This is because it helps to ensure that the project is aligned with the needs of the stakeholders, and that the development team is aware of the scope of the project.
- To improve the quality of the software: A well-defined and well-documented set of requirements can help to improve the quality of the software. This is because it helps to ensure that the software is designed and developed correctly, and that it meets the needs of the users.
- To save time and money: A well-defined and well-documented set of requirements can help to save time and money. This is because it helps to avoid rework, and it ensures that the development team is working on the right things.
-

❖ Technique of requirement elicitation

There are 4 technique of requirement elicitation

a. Traditional technique

- Introspection
- Reading existing document
- Analyze hard data
- Interview
- Survey/questioner
- Meeting

- b. Collaborative technique
 - Group technique
 - RAD work shop
 - Prototype
 - Participatory design
- c. Cognitive technique
 - Task analysis
 - Protocol analysis
 - Knowledge acquisition technique
- d. Contextual approach
 - Ethnographic technique
 - Discourse analysis
 - Sociotechnical method

Questionnaires: Questionnaires are a written survey that can be used to gather information from stakeholders. They can be a good way to gather information from a large number of stakeholders, but they may not be as detailed as other techniques.

Prototyping: Prototyping is the process of creating a working model of the system. This can be a good way to get feedback on the requirements from the stakeholders, and to see how they interact with the system.

Observation: Observation is the process of watching the stakeholder interact with the current system or process. This can be a good way to understand the stakeholder's needs and requirements in their own context.

Focus groups: Focus groups are a group discussion with a small number of stakeholders. They can be a good way to get feedback on the requirements from a variety of stakeholders.

Interviews: Interviews are a one-on-one conversation between the business analyst and the stakeholder. They can be a good way to gather detailed information about the stakeholder's needs and requirements.

Brainstorming: Brainstorming is a group technique in which participants generate ideas in a free-flowing and unstructured manner. It can be a useful technique for eliciting requirements from stakeholders, as it allows them to come up with new ideas and perspectives.

❖ Assess requirement document

- **Completeness:** Does the document contain all of the requirements that are needed to build the product or system?
- **Accuracy:** Are the requirements accurate and free of errors?
- **Clarity:** Are the requirements clear and easy to understand?
- **Consistency:** Are the requirements consistent with each other?
- **Traceability:** Can the requirements be traced back to the stakeholders' needs?
- **Verifiability:** Can the requirements be verified to ensure that they are met?
- **Usability:** Are the requirements usable by the stakeholders?
- **Maintainability:** Are the requirements maintainable so that they can be changed as needed?
- Who are the stakeholders for this project?

Requirement Eng. And SW architecture, design

- What are the goals of this project?
- What are the specific requirements for this project?

❖ Requirement management and tool

Management

Requirements management is the process of defining, documenting, and managing the requirements of a product or service. It is an essential part of any project, as it helps to ensure that the final product meets the needs of the users and stakeholders.

Tool

A requirements management tool is a software application that helps to automate the requirements management process. It can be used to store and track requirements, manage change requests, and collaborate with stakeholders

- **Xebrio:** The Most Adaptable and Flexible Requirements Management Tool
- IBM Rational DOORS - is a requirement management tool that helps organizations stay in control of their projects by providing an organized platform for gathering, storing, and managing requirements data.
- This software solution offers a collaborative environment where teams can work together to identify, describe and prioritize project requirements.
- Jama Software — Best for Software Development
- Visure — Best for Requirements Versioning Capabilities
- Modern Requirements4DevOps — Best for Azure DevOps Project Collaboration
- Orcanos — Best for Full QMS or ALM Solutions
- Helix RM — Best for Tool Integrations and Software Testing
- Jira -
- Rational door – used for optimizing, collaborating, and verifying requirements across the organization.
- **IRIS** – allows business owners to plan, deliver, and implement their corporation's strategies. It also provides rich design, modelling, and management functionality to track changes instantly.
- **Accompa** - It helps users to capture, track and manage requirements for any products & projects
- **Innoslate** - It allows users to import or create requirements documents easily. They can edit and review requirements directly in requirement view.
- **Polarion** – It gathers, approves and manage requirements for complex systems during the project lifecycle
- **Gatherspace** - is one of the first hosted on-demand web-based requirements management solutions. It helps to address the problems of project management failures.
- **Enterprise** - It integrates seamlessly with other development tools by creating requirements in the model
- **Tosca Testsuite** – It accelerates testing to keep pace with Agile and DevOps. It uses innovative, functional testing technologies.
- **iPlan** – it helps organizations to manage the entire project lifecycle. This tool also helps organizations to plan their projects and monitor its processes.
- **RaQuest** - It can be used for managing the requirements of systems and applications effectively. It is an ideal tool for the enhanced software testing and analysis.

Requirement Eng. And SW architecture, design

- **Cradle** - It is user-definable, scalable, flexible and secure. It can be deployed locally to remote sites. It can deliver through SaaS from any private or public cloud.
- **Helix RM** - It allows users to centralize and manage requirements, test cases, issues, etc.
- **Agile Designer** -It is a useful tool to reduce manual testing efforts. It allows organizations to deliver quality software at less price
- **CodeBeamer** - It also has many ALM Management capabilities. It can associate with source code, tasks, bugs, tests, releases and all other artifacts. It offers gapless traceability
- **ReqView** - is a simple but powerful requirements management tool. It allows capturing requirements in structured documents. It collaborates offline with the team by storing project data on a shared network drive.
- **Topteam analyst** - It converts textual requirements into easy-to-understand diagrams. This tool helps to understand requirements without reading too many documents.
- **Case Spec** - It helps users to manage any project artifact with complete traceability. It allows tracing requirements through design, testing, impact analysis, and gap analysis.
- **VersionOne** - This requirement management tool simplifies product planning with easy backlog management. This tool provides stakeholders with a centralized view of the backlog. It has a drag-and-drop interface.
- **Dimensions RM** - It helps to increase visibility and collaboration across business and delivery teams. It offers powerful reporting, tracking and provide end-to-end tractability
- **Atlassian Jira** - It helps every business
- **Silkroad Alm** - It has interlinking routes for the various legacy management system. It is a multi-purpose software which can act as requirement and test management tool.
- **Reqchecker** - It provides support to checks the coverage of the requirements between multiple documents. It also gives the benefit of requirement engineering to projects.
- **RMTrak** - is easy to use management tool for tagging, tracking, allocating, and verifying requirements. It is ideal software for anyone who wants simple and easy-to-use requirements tool for their organization.
- **TargetProcess** - is a visual requirement management and project management software solution. It focuses on the agile software development. It can be customized to provide support for customizing project manage
- **ReQtest** - is a cloud-based requirement management tool which offers end-to-end traceability. It can be your preferred ALM tool while working with requirements such as business, marketing, functional, non-functional, UI or any other requirements
- **Jama Software** - provides the leading platform for requirements, risk, and test management. With Jama Connect and industry-focused services, teams building complex products, systems, and software improve cycle times, increase quality, reduce rework, and minimize effort proving compliance.
- **Visure** - Solutions, Inc. is a trusted specialized requirement ALM partner for companies of all sizes across safety-critical and business-critical industries.
- **Modern Requirements** - which provides a collaborative Requirements Management Solution as a built-in extension to Microsoft's Azure DevOps, TFS or VSTS. The software reduces time to value delivery by providing users with end to end traceability, ingenious process automation, online document authoring, visualization support.

Software architecture & design

Requirement Eng. And SW architecture, design

❖ Software design principle

- Software design should correspond to the analysis model
- Choose the right programming paradigm
- Software design should be uniform and integrated
- Software design should be flexible
- Software design should ensure minimal conceptual (semantic) errors
- Software design should be structured to degrade gently
- Software design should represent correspondence between the software and real-world problem
- Software reuse
- Designing for testability
- Prototyping
- The design should be assessed for quality as it is being created, not after the fact.
- Design is not coding, coding is not design
- The design should be structured to degrade gently, even when aberrant data, events, or operating conditions are encountered.
- The design should be structured to accommodate change
- The design process should not suffer from "tunnel vision."
- The design should not reinvent the wheel.

Software design principles are guidelines that can be used to improve the quality of software. They are not specific solutions to problems, but rather general principles that can be applied to a wide variety of situations.

Some of the most important software design principles include:

- The Single Responsibility Principle (SRP) states that each class should have a single responsibility. This means that each class should only do one thing, and it should do that thing well.
- The Open-Closed Principle (OCP) states that software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification. This means that software should be designed in such a way that it can be easily extended with new features, without having to change the existing code.
- The Liskov Substitution Principle (LSP) states that any object that can be substituted for another object without affecting the correctness of the program should be substitutable for that object. This means that if you have a class that inherits from another class, the child class should be able to replace the parent class without breaking the program.
- The Interface Segregation Principle (ISP) states that clients should not be forced to depend on interfaces that they do not use. This means that if you have an interface with a lot of methods, you should consider splitting it into multiple interfaces, each with a smaller set of methods. This will make it easier for clients to only depend on the interfaces that they need.
- The Dependency Inversion Principle (DIP) states that high-level modules should not depend on low-level modules. Instead, both high-level and low-level modules should depend on abstractions. This means that you should design your software in such a way that the high-level modules do not depend on the implementation details of the low-level modules.

These are just a few of the many software design principles that can be used to improve the quality of software. By following these principles, you can create software that is more maintainable, extensible, and robust.

Requirement Eng. And SW architecture, design

Here are some additional software design principles that you may find helpful:

- Cohesion - The degree to which the elements of a module belong together.
- Coupling - The degree to which two modules depend on each other.
- Abstraction - The process of hiding implementation details from users.
- Encapsulation - The combination of data and methods into a single unit.
- Polymorphism - The ability of an object to take on different forms.

These principles can help you to create software that is easier to understand, maintain, and extend.

❖ Design pattern in software development

Software design pattern is a general, reusable solution of how to solve a common problem when designing an application or system.

Design patterns are typical solutions to commonly occurring problems in software design. The pattern typically shows relationships and interactions between classes or objects. It's not mandatory to always implement design patterns in your project. Design patterns are not meant for project development. Design patterns are meant for common problem solving.

i.e., Patterns = Problems + Solution pairs in a context

Design patterns are used to support object-oriented programming.

- Element of design pattern – there are four element
 - a. Pattern name
 - b. Problem
 - c. Solution
 - d. Consequences
- Usage of design pattern
 - a. To create common platform
 - b. Best practice (proven solution, reusable, expressive, prevent the need for refactoring code, lower the size of the codebase)
- Classification of design pattern

Table 1. Type of design patterns

Scope	Class	Purpose		
		Creational	Structural	Behavioral
	Class	Factory Method	Adapter (class)	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter (object) Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

- Based on purpose, level of detail, complexity, and scale of applicability to the entire system design pattern classified

Requirement Eng. And SW architecture, design

- a. Creational - deals with object creation and initialization, providing guidance about which objects are created for a given situation
 - Factory method – Creates objects with a common interface and lets a class defer instantiation to subclasses. provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created
 - Builder – aims to “Separate the construction of a complex object from its representation so that the same construction process can create different representations.” helps in minimizing the number of parameters in the constructor
 - Prototype – refers to creating duplicate object while keeping performance in mind.
 - Singleton - provides one of the best ways to create an object. n involves a single class which is responsible to create an object while making sure that only single object gets created.
 - b. Structural - deals with class and object composition, or how to assemble objects and classes into larger structures.
 - Adapter - works as a connector between two incompatible interfaces.
 - Façade - helps provide one interface (class) for access to a large body of code / various objects. It hides complexities of various sub-systems (often organized into a class) with a simple interface.
 - Bridge - A method to decouple an interface from its implementation.
 - Composite - Leverages a tree structure to support manipulation as one object. Is used where we need to treat a group of objects in similar way as a single object.
 - c. Behavioral - identifying common communication patterns between objects and realizing these patterns or how responsibilities are assigned between objects.
 - Command - is a data driven design pattern and falls under behavioral pattern category.
 - Interpreter - Provides a way to evaluate language grammar or expression. Is used in SQL parsing, symbol processing engine etc.
 - Mediator - Articulates simple communication between classes. Is used to reduce communication complexity between multiple objects or classes.
 - Observer - ” linking an object (subject) to dependents (observers) in a one-to-many pattern
 - Memento - A process to save and restore the internal/original state of an object. is falls under behavioral pattern category and used to restore state of an object to a previous state
- Based on scope it classify class and object

A software design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. It is not a finished design that can be transformed directly into source or machine code. Rather, it is a description or template for how to solve a problem that can be used in many different situations. Design patterns are formalized best practices that the programmer can use to solve common problems when designing an application or system.

Design patterns may be viewed as a structured approach to computer programming intermediate between the levels of a programming paradigm and a concrete algorithm.

There are three main types of design patterns:

- Creational patterns deal with the creation of objects.
- Structural patterns deal with the way objects are composed together.
- Behavioral patterns deal with the way objects interact with each other.

Some of the most popular design patterns include:

Requirement Eng. And SW architecture, design

- Singleton pattern ensures that only a single instance of a class is created.
- Factory pattern provides a way to create objects without specifying their concrete class.
- Adapter pattern allows objects with incompatible interfaces to work together.
- Strategy pattern allows you to dynamically change the algorithm used to perform a task.
- Observer pattern allows objects to subscribe to changes in other objects.

Design patterns can be used to improve the quality of software by making it more:

- Evolvable - Design patterns can help to make software more evolvable by making it easier to add new features or change existing ones.
- Maintainable - Design patterns can help to make software more maintainable by making it easier to understand and debug.
- Robust - Design patterns can help to make software more robust by making it more resistant to errors.
- Scalable - Design patterns can help to make software more scalable by making it easier to add new functionality or handle increasing loads.

When using design patterns, it is important to choose the right pattern for the problem at hand. There is no one-size-fits-all solution, and the best pattern will vary depending on the specific requirements of the system.

It is also important to understand the limitations of design patterns. Design patterns are not a silver bullet, and they cannot solve all problems. However, they can be a valuable tool for improving the quality of software.

❖ Architectural style and quality attribute

Architectural style

Software architecture styles are set of rules, constraints, or patterns of how to structure a system into a set of components and connectors.

- A. Pipe and filter –
 - used to structure systems which produce and process a stream of data
 - process data received from a set of inputs and send results to other subsystems via a set of outputs. Pipes can be used for buffering or for synchronization purposes.
- B. Client server
 - a system that hosts, delivers, and manages most of the resources and services that the client requests
 - all requests and services are delivered over a network,
- C. Peer to peer
 - It is a generalization of the client/server architecture in which subsystems can act both as client or as servers, in the sense that each subsystem can request and provide services.
- D. Event based
- E. Layering
- F. MVC (model view controller)

Requirement Eng. And SW architecture, design

- The view manages the graphical and/or textual output to the portion of the bitmapped display that is allocated to its application. It displays the information to the user.
- The controller interprets the mouse and keyboard inputs from the user, commanding the model and/or the view to change as appropriate. It handles the input from the user.
- The model manages the behavior and data of the application domain, responds to requests for information about its state (usually from the view), and responds to instructions to change state (usually from the controller). Model: contains the core functionality, data, the business logic, rules, and strategies.

G. SOA(service oriented architecture)

Quality attribute

Quality attributes describe externally visible properties of a software system and the expectations for that system's operation. Quality attributes define how well a system should perform some action. Sometimes called quality requirements

Architectural style and quality attribute are two important concepts in software design and architecture.

- Architectural style is a set of characteristics and features that make a software system notable or historically identifiable. It describes the component types and their topology, as well as the pattern of data and control interaction among the components.
- Quality attribute is a property of a software system that describes its externally visible characteristics and the expectations for that system's operation. Some examples of quality attributes include:
 - Reliability - The ability of a system to perform its intended functions under stated conditions for a specified period of time.
 - Usability - The ease with which a system can be used by its intended users.
 - Performance - The speed and efficiency with which a system can perform its tasks.
 - Scalability - The ability of a system to handle increasing loads or demands.

The relationship between architectural style and quality attribute is that architectural styles can be used to achieve specific quality attributes. For example, a layered architectural style can be used to achieve high performance, while a peer-to-peer architectural style can be used to achieve high scalability.

When designing a software system, it is important to consider the desired quality attributes and then select an architectural style that is well-suited to achieving those attributes.

Here are some examples of how architectural styles can be used to achieve specific quality attributes:

- Layered architectural style can be used to achieve high performance by decomposing the system into layers, with each layer performing a specific function. This can help to reduce the complexity of the system and improve communication between the layers.
- Peer-to-peer architectural style can be used to achieve high scalability by allowing components to communicate directly with each other, without the need for a central server. This can help to improve the performance of the system as the load increases.

Requirement Eng. And SW architecture, design

- Micro services architectural style can be used to achieve high flexibility by decomposing the system into small, independent services. This can help to make the system easier to maintain and update.

The choice of architectural style will depend on the specific requirements of the software system and the desired quality attributes. However, by carefully considering the relationship between architectural style and quality attribute, software architects can design systems that are both high-performing and reliable.

❖ Constructing and evaluate architecture

Constructing and evaluating architecture is a complex process that involves a number of steps. Here are some of the key steps involved:

1. Define the requirements. The first step is to define the requirements for the system. This includes identifying the stakeholders, their needs, and the constraints on the system.
2. Select an architectural style. Once the requirements have been defined, the next step is to select an architectural style. This is a set of principles and guidelines that can be used to design the system.
3. Design the architecture. The next step is to design the architecture of the system. This involves identifying the components of the system, their interactions, and the deployment environment.
4. Evaluate the architecture. Once the architecture has been designed, it is important to evaluate it. This involves assessing the architecture against the requirements and identifying any potential problems.
5. Iterate. The process of constructing and evaluating architecture is iterative. This means that the steps may need to be repeated several times as the design of the system evolves.

Here are some of the key factors to consider when evaluating architecture:

- Alignment with requirements. The architecture should be aligned with the requirements of the system.
- Robustness. The architecture should be robust and able to withstand unexpected events.
- Scalability. The architecture should be scalable and able to handle increasing loads.
- Maintainability. The architecture should be maintainable and easy to update.
- Cost-effectiveness. The architecture should be cost-effective and meet the budget constraints.

By carefully considering these factors, architects can design systems that are both effective and efficient.

Here are some of the tools that can be used to construct and evaluate architecture:

- UML (Unified Modeling Language) is a graphical language that can be used to model the architecture of a system.
- Architectural frameworks provide a set of predefined components and patterns that can be used to design the architecture of a system.
- Simulation tools can be used to simulate the behavior of a system and evaluate its performance.